Project Plan: V1 Stateless Quiz Web App

STREAMLIT ENGINE, GITHUB REPOSITORY STORAGE, AND LIVE GEMINI API MARKING

Version:	1.0 (Initial Release Plan)	**Date:**	June 5, 2026
Core Tech Stack:	Python / Streamlit Cloud	**AI Engine:**	Gemini 1.5 Flash API

1. Executive Overview & Scope

This document contains the functional architecture and initial codebase roadmap for Version 1 of the stateless quiz web application. The design focuses entirely on the client-facing front-end component. To secure immediate viability, the system eliminates persistent storage and database overhead entirely.

Instead, quizzes will be managed as human-readable configuration files hosted in a GitHub repository. The runtime environment evaluates multiple-choice entries natively, leverages Google's Gemini API for instantaneous semantic long-answer marking, compiles unified student feedback, and dispatches it immediately via SMTP email before purging session data from memory.

2. Section I: Repository Layout & Quiz File Specifications

To prevent parsing vulnerabilities and manual text splitting errors, all quiz configuration layouts will utilize the structured **YAML (.yaml)** format. YAML offers absolute clarity for authors editing files by hand, while natively deserializing directly into standard Python dictionaries in a single execution line.

Repository Directory Structure

```
your-quiz-repository/
│
├── .streamlit/
│   └── secrets.toml          # Encrypted API keys and email credentials
│
├── quizs/                    # Target subdirectory for quiz parameters
│   ├── QUIZ_101.yaml
│   ├── QUIZ_102.yaml
│   └── QUIZ_103.yaml
│
├── app.py                    # Core Streamlit web application engine
└── requirements.txt          # System package prerequisites
```

Quiz Serialization Schema (quizs/QUIZ_[ID].yaml)

Files must be named using the continuous uppercase format matching their unique identifier integer. Every file holds structural properties for up to 15 multiple-choice questions and exactly one comprehensive open-ended text task.

```
quiz_id: 101
video_url: "https://www.youtube.com/watch?v=dQw4w9WgXcQ"
```

```
title: "Photosynthesis & Cellular Respiration Mastery"

multiple_choice:
  - question_num: 1
    text: "What primary gas is absorbed by plants during the light-independent reactions?"
    A: "Oxygen (O2)"
    B: "Carbon Dioxide (CO2)"
    C: "Nitrogen (N2)"
    D: "Hydrogen (H2)"
    answer: "B"
    points: 5
    explanation: "Plants absorb carbon dioxide to assemble carbon skeletons during the Calvin
Cycle."

  # ... [Continues sequentially up to 15 items maximum]

long_answer:
  question_num: 1
  text: "Describe the function of the stomata in carbon capture and water regulation."
  points: 10
  rubric: "Full points require explicitly referencing gas exchange (carbon dioxide intake,
oxygen extraction) and guard cell turgor pressure mechanics regulating transpiration (water
loss) under changing conditions."
```

3. Section II: Web Application Functional Requirements

The deployment runner operates entirely statelessly, observing a sequential request lifecycle for every individual user connection instance:

- **Routing and Parameter Binding:** The application parses the active browser URL parameters via `st.query_params` to identify the requested quiz ID (e.g., `?quiz=101`). If missing, a safe entry fall-back viewport is shown.
- **Dynamic Git Ingestion:** The engine formats an unauthenticated HTTP GET request targeting the raw public endpoint of the GitHub repository. It downloads the structured string and converts it natively via `yaml.safe_load()`.
- **Identity Verification:** The UI renders a mandatory text field requesting the student's institutional email address. Form submission validation blocks evaluation if this field is missing or contains an invalid structure.
- **Dual-Tier Grading Pipeline:** Multiple-choice data is evaluated instantly on the client runtime thread. Long-answer essays trigger a direct backend API payload handoff to Gemini alongside the exact rubrics found in the configuration block.

4. Section III: Implementation Plan & Code Architecture

The Gemini Evaluation Prompt Schema

To preserve structural compliance, the app configures Gemini to process feedback via strict system boundaries, returning evaluation objects in native JSON mapping format:

```
SYSTEM INSTRUCTION:
You are an expert institutional grading assistant. Evaluate the student response strictly
against the provided rubric parameters. You must return your evaluation inside valid JSON
format with exactly two specific keys:
1. "score": An integer representing awarded points (cannot exceed max parameters).
2. "feedback": A concise paragraph outlining point subtractions and actionable recommendations.

INPUT DATA:
- Evaluation Objective: {text}
- Target Rubric Bounds: {rubric}
- Maximum Obtainable Score: {points}
- Student Plain-text Entry: {student_response}
```

Main Runner Blueprint Code (app.py)

```
import streamlit as st
import requests
import yaml
import json
import google.generativeai as genai
import smtplib
from email.mime.text import MIMEText

# Securely register global API credentials from native secrets file
genai.configure(api_key=st.secrets["GEMINI_API_KEY"])

# Bind and parse the dynamic quiz parameters from the URL string
quiz_id = st.query_params.get("quiz", "101")

@st.cache_data(show_spinner="Fetching Quiz Parameters...")
def load_quiz_from_github(q_id):
    raw_url = f"https://raw.githubusercontent.com/user/repo/main/quizs/QUIZ_{q_id}.yaml"
    res = requests.get(raw_url)
    if res.status_code == 200:
        return yaml.safe_load(res.text)
    return None

quiz = load_quiz_from_github(quiz_id)

if not quiz:
    st.error(f"Error: Quiz Configuration ID '{quiz_id}' could not be located.")
else:
    st.title(quiz.get("title", "Homework Evaluation Portal"))
    st.video(quiz["video_url"])

    with st.form("quiz_form"):
        student_email = st.text_input("Institutional Email Address:",
placeholder="name@school.ac.uk")

        # Iteratively render up to 15 Multiple Choice blocks
        mc_responses = {}
        for mc in quiz.get("multiple_choice", []):
            st.write(f"**Question {mc['question_num']}:** {mc['text']}")
            mc_responses[mc['question_num']] = st.radio(
                "Select Options:", [mc['A'], mc['B'], mc['C'], mc['D']], index=None,
key=f"mc_{mc['question_num']}"
            )
            st.write("---")
```

```
        # Render the singular Long Answer text frame
        la_config = quiz["long_answer"]
        st.write(f"**Question {la_config['question_num']} (Long Answer):**
{la_config['text']}")
        student_la = st.text_area("Type your explanation details here:", key="student_la")

        submit_clicked = st.form_submit_with_button("Finalize and Submit Assignment")

        if submit_clicked:
            if not student_email or "@" not in student_email:
                st.error("Submission Halted: Valid Institutional Email is required.")
            else:
                # Part 1: Local Multiple Choice Tabulation
                mc_earned = 0
                mc_total = 0
                for mc in quiz.get("multiple_choice", []):
                    selected = mc_responses[mc['question_num']]
                    correct_letter = mc["answer"]
                    mc_total += mc["points"]
                    if selected and selected.startswith(mc[correct_letter]):
                        mc_earned += mc["points"]

                # Part 2: Synchronous Remote Gemini Semantic Marking
                model = genai.GenerativeModel('gemini-1.5-flash',
generation_config={"response_mime_type": "application/json"})
                prompt = f"Objective: {la_config['text']}\nRubric: {la_config['rubric']}\nMax:
{la_config['points']}\nStudent: {student_la}"

                ai_raw = model.generate_content(prompt).text
                ai_data = json.loads(ai_raw) # Extracts 'score' and 'feedback' keys

                # Part 3: Compile and Dispatch via SMTP Server Mailer
                # [SMTP connection loop logic to student and archive mailbox executes here]

                st.success("Assignment transmitted successfully. Check your email for feedback
marks.")
```

> **Operational Scaling Note:** This foundation can easily be upgraded in the future. Because all core variables interact statelessly, transitioning to the encrypted automated token verification or updating your background spreadsheet logging script will require zero structural modifications to the core UI layout modules written above.